

Übungsblatt 2: 3D-NPC mit KI-gesteuertem Verhalten

1. Wahl der Engine - Godot

Zur Bewältigung dieser Aufgabe entschied ich mich recht früh insofern vom Aufgabenblatt abzuweichen, dass ich für diese Aufgabe eine von Grund auf neues Projekt erstelle, statt auf dem Labyrinth aus Aufgabe 1 aufzubauen. Dafür sprachen für mich vor allem die folgenden 2 Gründe:

1. Ich will gerne mehr Erfahrung mit den verschiedenen Game-Engines sammeln. Während ich die erste Aufgabe in der Unity Engine löste, wollte ich dieses Mal Godot als Open-Source Alternative ausprobieren. Vor allem wie gut das Arbeiten mit GDScript als eigene, aufs Programmieren von Spielen fokussierte Programmiersprache, im Vergleich zu C# funktioniert, möchte ich herausfinden. Neben der Wahl der Programmiersprache trifft Godot zudem einige sympathische Entscheidungen, z. B. die Wahl von Vulkan als Grafik-API und der Umstand, dass die gesamte Engine mit allen Abhängigkeiten (für Windows) in nur ca. 90MB ausgeliefert wird.
2. Das Verlassen des Labyrinths bietet viel mehr Interaktionsmöglichkeiten mit dem NPC, die in einem Labyrinth fehl am Platz gewirkt hätten.

Der Einstieg in die neue Engine mit dem Vorwissen von Unity war sehr simpel. Generell scheinen beide Engines hinsichtlich grundlegenden Workflow und essentiellen Tools sehr ähnlich zu sein. Das Arbeiten mit 3D-Primitiven für schnelle Prototypen scheint in Godot komplizierter zu sein, da es mehr verschiedene Typen gibt (MeshRenderer, CSG, StaticBody, ...), welche in unterschiedlichen Kombinationen mit den Kollisionsprimitiven CollisionShape3D oder CollisionPolygon3D kompatibel sind, oder auch nicht. Beispielsweise war mir nicht ersichtlich, weshalb ich nicht direkt einer CSGBox3D mittels CollisionShape3D eine Kollision verleihen konnte. Stattdessen musste ich einen StaticBody3D als Elternknoten für die CSGBox3D erstellen und dann die CollisionShape3D dem StaticBody3D hinzufügen.

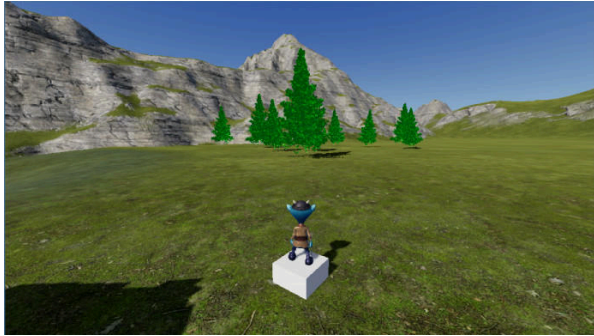
2. Umgebung - Die Heimat des NPCs

Als Umgebung wollte ich in dieser Übung ein größeres Terrain nutzen, welches der NPC bewohnt. Da Godot leider nativ keine Tools zur Erstellung von Terrains bietet, habe ich auf das Terrain3D-Plugin¹ zurückgegriffen. Während es auf dem Papier um ein „high performance“ Terrain System für Godot handelt, musste ich leider mit vielen Problemen und Bugs kämpfen.

- Während anfangs die simplen Tools zum Erhöhen und Erniedrigen des Terrains funktionierten, kam es irgendwann recht zuverlässig beim Editieren zu Crashes, sodass ich mich letztlich entschied, das mitgelieferte Demo-Terrain statt einem eigenen Terrain zu nutzen.
- Es scheint ein Problem bei der Kollisionsberechnung mit dem Terrain zu geben, sodass ich einige Male durch das Terrain hindurchfiel.
- Terrain3D unterstützt Instance Meshes, um Vegetation und andere Objekte zu platzieren. Es scheint sogar ein Feature integriert zu sein, das diese Meshes unterschiedlich rendert, je nachdem, wie weit die Kamera vom Objekt entfernt ist. Dies scheint jedoch genau falsch

¹<https://github.com/TokisanGames/Terrain3D>

herum zu funktionieren - wenn der Spieler sich einem Baum nähert, verschwinden aus mysteriösen Gründen seine Blätter.



Nichtsdestotrotz hatte ich mit Terrain3D nach kurzer Zeit eine zumindest nutzbare Welt, die ich anschließend etwas mit Leben füllen konnte. Damit der NPC an einer kleinen Hütte am See wohnen konnte, versuchte ich mich etwas an der Erstellung eines Wassershaders und der Einbindung von Umgebungsgeräuschen. Die verwendeten Ressourcen wie Modelle, Animationen, etc. finden sich wieder in der Datei `credits.md`.

3. Interaktionen mit dem NPC

Generell kann mein NPC einen von 8 Zuständen sein: `IDLE`, `SCARED`, `ANGRY`, `ATTACKING`, `DEAD`, `CHATTING`, `BLOCKING`, `COUNTERING`.

Nachfolgend beschreibe ich zunächst meine Ideen, wie man ein LLM für das NPC-Verhalten gewinnbringend einsetzen könnte, um die NPC Zustände zu implementieren. Die erste Idee erfordert dabei wirklich ein LLM. Die weiteren Ideen könnte man sicherlich auch logisch programmieren, oder mit einem sehr simplen Machine Learning Ansatz (z. B. kleines neuronales Netzwerk) lösen. Hier sehe ich den Vorteil der Verwendung eines LLMs vor allem darin, dass nur ein Prompt geschrieben werden muss und die Programmierarbeit dadurch minimiert wird. Dies beschleunigt sicherlich die Entwicklung und erlaubt es auch Experten ohne Programmierkenntnis die Interaktionen nach ihren Vorstellungen zu gestalten.

3.1. Dialog

Projiziert auf ein richtiges Spiel wäre meine Vision, nur den Charakter eines NPCs in einem Prompt zu beschreiben, und mithilfe des LLMs einen dynamischen Dialog zu generieren, welcher sowohl auf den unmittelbaren Zustand des Spielers eingehen sollte, aber sogar zusätzliche Echtzeitinformationen einbeziehen könnte. Interessante Beispiele, die mir dazu einfallen:

- Einbeziehen der Uhrzeit: „Du besuchst mich zu so einer Uhrzeit, und dann auch noch an einem Sonntag?!“
- Bezug nehmen auf aktuelle Ereignisse, z. B. könnte ein Fußballfan versuchen, mit dem Spieler den Verlauf des letzten Spiels seiner Lieblingsmannschaft zu diskutieren.
- Bezug nehmen auf aktuelle Weltereignisse in einem MMO Kontext, z. B. könnte der NPC der gesamten Spielerschaft gratulieren, einen Planeten befreit zu haben (vgl. Helldivers²).

Insbesondere könnte dieser Ansatz es erlauben, dass sich NPCs in einem Spiel viel lebendiger anfühlen. Zudem sollte dazu in der Lage sein, recht elegant das häufige Immersionsproblem, dass der Spieler beliebige virtuelle Gräueltaten verüben kann, ohne dass dies einen Einfluss auf sein Verhältnis mit den NPCs hat, verhindern.

²<https://gameme.eu/wie-man-in-helldivers-2-planeten-fuer-grossauftraege-befreit/>

Dazu müsste man dem LLM einen umfassenden Kontext übergeben, der sämtliche benötigten Infos dafür einhält. Dabei muss man stets abwägen, dass zusätzliche Infos definitiv die Verarbeitungszeit und damit die letztendliche Latenz erhöhen können.

3.2. Logische Minispiele

Eine weitere Chance bieten LLM für die Implementierung von (möglichst optimalem) Gegnerverhalten in Logikspielen wie Schere-Stein-Papier oder dem Nim-Spiel (bei dem wir ja schon in der Vorlesung gesehen haben, dass ein LLM durchaus dazu in der Lage ist, optimal zu spielen).

Für das Nim-Spiel würde es reichen, nur die Anzahl der Steichhölzer (je nach Spielart: pro Reihe) zu übertragen, was den Kontext ausreichend kurz für schnelle Antworten halten sollte.

Für das Schere-Stein-Papier-Spiel, welches ich auch in meinem Spiel implementiert habe, ist es interessant, dem LLM die vergangenen k Spielzüge des Spielers mitzuteilen, um so absichtliche oder unabsichtliche Regelmäßigkeiten des Spielverhaltens des Spielers auszunutzen.

Hierbei ist mir jedoch aufgefallen, dass das Verlängern des Prompts um zusätzliche Anweisungen (die nur mit dem Format zu tun hatten), schnell zu einem deutlich schlechteren Verhalten des LLM führt. Selbst bei größeren Modellen, wie Googles Gemini 2.5 Flash Lite, was ich verwende, scheinen Details im Prompt schnell vergessen zu werden. Das führt dazu, dass bei einer Schere-Stein-Papier-Anfrage zum Glück immer ein Schere-Stein-Papier-Zug zurückgegeben wird, das LLM jedoch fast aktiv versucht, so schlecht wie möglich zu spielen (trotz der Anweisung optimal zu spielen).

3.3. Kampfsystem

Mit einem ähnlichen Ansatz wie für die logischen Minispiele kann man auch ein simples Kampfsystem umsetzen. Der Spieler und der NPC haben dabei 20 HP und beide Zugriff auf die folgenden 3 Aktionen:

1. Angriff. Fügt dem Gegenüber 5 HP Schaden zu.
2. Blocken. Blockt 3HP Schaden des Gegners, sodass dieser maximal 2HP Schaden (bei Angriff) bzw. 0 bei Parieren zufügen kann.
3. Parieren. Greift das Gegenüber gerade an, blocke den Schaden und verursache ihm 10 HP Schaden. Ansonsten erleide selbst 2 HP Schaden.

Daraus ergibt sich, dass Parieren optimal gegen Angreifen ist, während Blocken optimal gegen Parieren und Angreifen optimal gegen Blocken ist. Ähnlich wie beim Schere-Stein-Papier-Spiel sollte ein LLM auch hier in der Lage sein, mit dem Wissen über die letzten k Aktionen des Spielers eine vernünftige Strategie zu entwickeln.

Allerdings kommt hier erschwerend hinzu, dass der NPC nach Beendigung der aktuellen Animation schon die nächste Aktion ausgewählt haben muss (sofern es sich nicht um eine rundenbasiertes Kampfsystem handelt). Deshalb habe ich mich dazu entschieden, bei jeder LLM-Anfrage die 10 nächsten NPC-Aktionen generieren zu lassen (**Pufferung**). Hat man Animationen von ca. 1s Dauer, sollte die Zeit reichen, damit die nächsten Aktionen generiert werden können, bevor der NPC ratlos in der Gegend herumstehen müsste. Um Kämpfe zu verlängern und dem LLM eine Chance zu geben, das Spielerverhalten zu analysieren, bevor der NPC tot ist, werden Verteidigungsaktionen vom Kampfsystem eher belohnt als das Angreifen.

Der Nachteil der Pufferung ist natürlich, dass der NPC immer erst bis zu 10s später auf das Verhalten des Spielers reagieren kann, für findige Spieler stellt dies wahrscheinlich keine Herausforderung dar (anfangs nur zu blocken sollte das LLM dazu provozieren, vornehmlich anzugreifen, woraufhin der Spieler permanent parieren sollte). Trotzdem könnte dieser Ansatz, in einer ausgefeilteren Iteration, eine interessante Alternative zu zufallsbasierten Verhaltensmustern sein.

4. Technische Details

Bezüglich des Zeitpunkts der Kommunikation mit dem LLM habe ich zwei Optionen ausprobiert:

1. Polling.

In regelmäßigen Abständen, z. B. alle 2 Sekunden, erhält das LLM die aktuelle Situation und liefert basierend darauf das Verhalten des NPCs. Dieser Ansatz ist besonders schön, da er die gesamte Entscheidungsgewalt und Logik an das LLM überträgt und keine festcodierten Regeln à la „be scared if player_distance < 10 and player has weapon equipped“ benötigt werden und man so der auf Discord besprochenen Lösung am nächsten kommt. Folgende Probleme sehe ich für eine tatsächliche Implementierung im großen Rahmen:

- *Bandbreiten- und Latenzanforderungen:* Nach einem kleinen Experiment, was ich durchgeführt habe, verursacht der Prompt ohne Historie etwa 568 Byte (inkl. JSON-Overhead), was in einem echten Spiel, durch eine deutlich genauere Charakterbeschreibung des entsprechenden NPCs und mehr Aktionen sicherlich auf über 2KB ansteigen könnte. Eine DIALOGUE-Antwort des LLM mit konservativen 50 Wörtern Text verursacht etwa 339 Byte. Dementsprechend würden bereits bei 1000 Spielern (auf, wenn es sich um ein Single-Player Spiel handelt) und 10 NPCs etwa 40 MB/s eingehend und 6 MB/s ausgehend (bzw. wenn ein externer LLM-Dienstleister verwendet wird, noch bedeutend mehr) anfallend. Diese problematische Skalierung wird weiter erschwert, da es nicht trivial ist, einen LLM-Anbieter zu finden, der bei steigenden Spielerzahlen immer eine Latenz < 1s erreicht. Gemini 2.5 Flash Lite, was extra auf „ultra-low latency“ optimiert ist, hatte in meinen Tests bereits bei einem einzigen Spieler eine Latenz von 1-5s.
- *keine Historie:* Aufbauend auf dem vorherigen Punkt ist es fast unmöglich, einem LLM bei jedem Aufruf eine Historie der vorherigen Zustände und Aktionen bereitzustellen, da dies die Probleme Bandbreiten- und Latenzprobleme noch verstärkt. Dadurch kann es aber zu sehr inkonsistentem Verhalten kommen, da das LLM nicht weiß, was der NPC vor einigen Sekunden getan hat.
- *Pollinghäufigkeit:* Abhängig davon, wie schnell das Spiel ist, müsste die Pollinghäufigkeit angepasst werden. Wenn sich der Spieler innerhalb von wenigen Sekunden über das gesamte Terrain bewegen kann, würde es sicherlich nicht reichen, wenn der NPC nur alle 2 Sekunden die Chance hat, sein Verhalten zu ändern. Bei der klassischen Programmierung wird das Verhalten oft in einer `FixedUpdate`-Methode implementiert, welche alle 0.02 Sekunden aufgerufen wird. Dieses Problem kann mitigiert werden, indem basierend auf der aktuellen Situation die Häufigkeit erhöht/reduziert wird. Beispielsweise reicht es wohl, für weit entfernte NPCs nur selten LLM-Anfragen verschickt, wohingegen dies für nahe NPCs (v. a. wenn sie sich im Kampf (siehe Abschnitt 3.3) befinden) häufiger passieren sollte.

2. Trigger-basierte LLM-Aufrufe.

Um die Anzahl der LLM-Aufrufe minimal zu halten und nur zu versenden, wenn die

Situation dies wirklich erfordert, ist es naheliegend, basierend auf festen Regeln eine Anfrage zu stellen. Somit ergibt sich die Reaktion des NPCs nicht ausschließlich durch das LLM, sondern kann als hybride Steuerung zwischen klassischer (Spiele-KI-)Programmierung und LLM-Steuerung gesehen werden. In einem Dialog ist es beispielsweise naheliegend, erst dann das Verhalten zu aktualisieren und zu antworten, wenn der Spieler seinen Text eingegeben hat. Ebenso sieht es bei Schere-Stein-Papier und im Kampf aus.

Um einen vernünftigen Mittelweg zu gehen, habe ich mich dazu entschieden, Polling einzusetzen, solange der NPC im `IDLE`, `BE_SCARED`, `BE_ANGRY`-Zustand ist (wobei die Dauer bis zum nächsten Poll berechnet wird als $\min\left\{\frac{\text{player_max_speed}}{\text{player_velocity}}, \frac{\text{player_distance}}{3}\right\} \times 3$). Wechselt er in einen der Zustände `SCHERE_STEIN_PAPIER`, `ATTACKING`, `DIALOGUE` werden hingegen nur trigger-basierte LLM-Aufrufe durchgeführt. Die Übergänge `Polling` $\rightarrow$ `trigger-based` und `trigger-based` $\rightarrow$ `Polling` können dabei vom LLM oder vom Spieler hervorgerufen werden.

4.1. LLM-Antwortformat und Animationen

Eine Anfrage an das LLM im Polling-Modus in der Funktion `npc.describe_situation` zusammengebaut. Enthalten sind neben dem Preprompt (beschreibt Kontext als hessischer NPC in einem Videospiel) die Entfernung des NPCs zum Spieler, Entfernung zu seiner Hütte, Entfernung zum See, Geschwindigkeit des Spielers und, ob der Spieler seine Waffe verwendet hat. Im Gespräch (verarbeitet in `npc.react_to_chat`) bleibt die Nachrichten-Historie erhalten, um zusätzlichen Kontext für das Gespräch zu bieten. Schere-Stein-Papier wird in `npc.react_ssp` verarbeitet und erhält alle bisherigen Spielzüge des Spielers als Kontext. Das Kampfsystem wird in `npc.process_fighting_round` verarbeitet. Als Antwortformat des LLM entschied ich mich in allen Fällen für dieses simple JSON-Format:

```
{
  "action": <ACTION>,
  "content": <CONTENT>
}
```

`ACTION` kann dabei einer der Zustände aus Abschnitt 3 sein. `CONTENT` soll nur bei `CHAT_WITH_PLAYER`, `SCHERE_STEIN_PAPIER` und `FIGHT_PLAYER` auf einen Text, einen Spielzug oder eine Liste von 10 Kampfzügen gesetzt sein. In meinen Experimenten funktionierte dies die meiste Zeit relativ zuverlässig. Um sicher zu gehen, ließe sich im Produktiveinsatz das Format mittels „Structured Outputs“³ erzwingen, wogegen ich mich aber zur einfacheren Austauschbarkeit von OpenRouter und Ollama entschieden habe.

Zu den vorhandenen `ACTIONS` existieren im Enum `State` zugehörige Zustände, in die der NPC dann versetzt wird. Im `AnimationTree` sind Animationswechsel an die jeweiligen Zustandswechsel gebunden. Handelt es sich um `CHATTING`, wird der im `CONTENT` befindliche Text via TTS ausgegeben.

Der NPC hat die folgenden Animationen: `IDLE`, `BE_SCARED`, `BE_ANGRY`, `FLYING`, `ATTACKING`, `BLOCKING`, `COUNTERING`, `TALKING`, `DYING`.

Der Spieler kann überdies noch `SPRINTEN`, `SPRINGEN`, `SCHWIMMEN` und hat je nach Geschwindigkeit und Zustand 3 verschiedene Angriffsanimationen.

³<https://openai.com/index/introducing-structured-outputs-in-the-api/>